



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
DE LA RECHERCHE SCIENTIFIQUE ET DE L'INNOVATION

UNIVERSITÉ SULTAN MOULAY SLIMANE

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
DE KHOURIBGA



TP : Book Reading Tracker

Encadré par :
Pr. OURDOU

Réalisé par :
Nassima RHANNOUCH

Année universitaire : 2024-2025

6 novembre 2025

Table des matières

1	Introduction	2
1.1	Objectifs du TP	2
1.2	Technologies utilisées	2
2	Question 1 : Création du formulaire d'enregistrement des livres	3
2.1	Énoncé	3
2.2	Implémentation	4
2.3	Capture d'écran du formulaire	7
3	Question 2 : Classe Book et ses méthodes	8
3.1	Énoncé	8
3.2	Implémentation	8
3.3	Capture d'écran du code	9
4	Question 3 : Page de suivi de lecture	10
4.1	Énoncé	10
4.2	Description de l'implémentation	10
4.3	Capture d'écran - Statistiques globales	10
4.4	Capture d'écran - Affichage des livres	11
5	Question 4 : Intégration avec MongoDB	12
5.1	Énoncé	12
5.2	Implémentation	12
5.3	Captures d'écran - Connexion MongoDB	15
6	Tests avec Postman	16
6.1	Introduction aux tests	16
6.2	Tests des différentes routes	16
6.2.1	Test 1 : GET /api/books - Récupérer tous les livres	16
6.2.2	Test 2 : POST /api/books - Ajouter un nouveau livre	16
6.2.3	Test 3 : PUT /api/books/ :title - Mettre à jour un livre	18
6.2.4	Test 4 : DELETE /api/books/ :title - Supprimer un livre	18
6.2.5	Test 5 : GET /api/stats - Récupérer les statistiques	19
7	Conclusion	20

1 Introduction

Ce rapport présente le travail réalisé dans le cadre du TP sur le développement d'une application web de suivi de lecture de livres (Book Reading Tracker). L'objectif de ce projet est de créer une application permettant aux utilisateurs d'enregistrer, gérer et suivre leur progression de lecture.

1.1 Objectifs du TP

Les principaux objectifs de ce travail pratique sont :

- Développer une interface web responsive utilisant HTML et TailwindCSS
- Implémenter un système de gestion de livres avec JavaScript/TypeScript
- Intégrer une base de données MongoDB pour la persistance des données
- Créer un tableau de bord de suivi de lecture avec statistiques
- Tester l'API avec Postman

1.2 Technologies utilisées

- **Frontend** : HTML5, CSS3 (TailwindCSS), JavaScript
- **Backend** : Node.js, TypeScript, Express
- **Base de données** : MongoDB
- **Outils de test** : Postman

2 Question 1 : Création du formulaire d'enregistrement des livres

2.1 Énoncé

Créer un fichier HTML contenant un formulaire pour enregistrer de nouveaux livres. Utiliser TailwindCSS pour le style. Chaque livre possède les attributs suivants :

- **title** (string) : Titre du livre
- **author** (string) : Auteur du livre
- **number of pages** (number) : Nombre total de pages
- **status** (String Enum) : Statut de lecture
- **price** (number) : Prix du livre
- **number of pages read** (number) : Pages lues (< nombre de pages)
- **format** (String Enum) : Format du livre
- **suggested by** (string) : Suggéré par
- **finished** (boolean) : Livre terminé (par défaut 0)

Valeurs possibles pour Status :

[noitemsep]

- Read (Lu)
- Re-read (Relu)
- DNF (Did Not Finish - Abandonné)
- Currently reading (En cours de lecture)
- Returned Unread (Retourné non lu)
- Want to read (À lire)

Valeurs possibles pour Format :

[noitemsep]

- Print (Papier)
- PDF
- Ebook
- AudioBook (Livre audio)

2.2 Implémentation

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Book Reading Tracker</title>
7   <script src="https://cdn.tailwindcss.com"></script>
8 </head>
9 <body class="bg-gray-100 min-h-screen">
10   <div class="container mx-auto px-4 py-8">
11     <h1 class="text-4xl font-bold text-center text-blue-600 mb-8">
12       Book Reading Tracker
13     </h1>
14
15     <!-- Section des statistiques globales -->
16     <div class="bg-white rounded-lg shadow-md p-6 mb-8">
17       <h2 class="text-2xl font-semibold mb-4 text-gray-800">
18         Statistiques Globales
19       </h2>
20       <div class="grid grid-cols-1 md:grid-cols-3 gap-4">
21         <div class="bg-blue-50 p-4 rounded-lg">
22           <p class="text-gray-600">Total de livres</p>
23           <p id="totalBooks" class="text-3xl font-bold text-
24             blue-600">
25             0
26           </p>
27         </div>
28         <div class="bg-green-50 p-4 rounded-lg">
29           <p class="text-gray-600">Livres terminés</p>
30           <p id="finishedBooks" class="text-3xl font-bold text-
31             -green-600">
32             0
33           </p>
34         </div>
35         <div class="bg-purple-50 p-4 rounded-lg">
36           <p class="text-gray-600">Pages totales lues</p>
37           <p id="totalPages" class="text-3xl font-bold text-
38             purple-600">
39             0
40           </p>
41         </div>
42       </div>
43
44     <!-- Formulaire d'ajout de livre -->
45     <div class="bg-white rounded-lg shadow-md p-6 mb-8">
46       <h2 class="text-2xl font-semibold mb-4 text-gray-800">
47         Ajouter un Livre
48       </h2>
49       <form id="bookForm" class="grid grid-cols-1 md:grid-cols-2
50         gap-4">
51         <div>
52           <label class="block text-gray-700 mb-2">Titre *</
53           label>
54           <input type="text" id="title" required

```

```

51         class="w-full px-4 py-2 border border-gray-300
rounded-lg
52         focus:outline-none focus:ring-2 focus:ring-blue
-500">
53     </div>
54     <div>
55         <label class="block text-gray-700 mb-2">Auteur *</
label>
56         <input type="text" id="author" required
57         class="w-full px-4 py-2 border border-gray-300
rounded-lg
58         focus:outline-none focus:ring-2 focus:ring-blue
-500">
59     </div>
60     <div>
61         <label class="block text-gray-700 mb-2">
62         Nombre de pages *
63         </label>
64         <input type="number" id="numberOfPages" required min
="1"
65         class="w-full px-4 py-2 border border-gray-300
rounded-lg
66         focus:outline-none focus:ring-2 focus:ring-blue
-500">
67     </div>
68     <div>
69         <label class="block text-gray-700 mb-2">Pages lues</
label>
70         <input type="number" id="pagesRead" value="0" min="0
"
71         class="w-full px-4 py-2 border border-gray-300
rounded-lg
72         focus:outline-none focus:ring-2 focus:ring-blue
-500">
73     </div>
74     <div>
75         <label class="block text-gray-700 mb-2">Statut *</
label>
76         <select id="status" required
77         class="w-full px-4 py-2 border border-gray-300
rounded-lg
78         focus:outline-none focus:ring-2 focus:ring-blue
-500">
79             <option value="Want to read">Want to read</
option>
80             <option value="Currently reading">Currently
reading</option>
81             <option value="Read">Read</option>
82             <option value="Re-read">Re-read</option>
83             <option value="DNF">DNF</option>
84             <option value="Returned Unread">Returned Unread<
/option>
85         </select>
86     </div>
87     <div>
88         <label class="block text-gray-700 mb-2">Format *</
label>
89         <select id="format" required

```

```

90         class="w-full px-4 py-2 border border-gray-300
rounded-lg
91         focus:outline-none focus:ring-2 focus:ring-blue
-500">
92         <option value="Print">Print</option>
93         <option value="PDF">PDF</option>
94         <option value="Ebook">Ebook</option>
95         <option value="AudioBook">AudioBook</option>
96     </select>
97 </div>
98 <div>
99     <label class="block text-gray-700 mb-2">Prix (EUR)</
label>
100     <input type="number" id="price" value="0" min="0"
step="0.01"
101         class="w-full px-4 py-2 border border-gray-300
rounded-lg
102         focus:outline-none focus:ring-2 focus:ring-blue
-500">
103 </div>
104 <div>
105     <label class="block text-gray-700 mb-2">Suggere par<
/label>
106     <input type="text" id="suggestedBy"
class="w-full px-4 py-2 border border-gray-300
rounded-lg
107         focus:outline-none focus:ring-2 focus:ring-blue
-500">
108 </div>
109 <div class="md:col-span-2">
110     <button type="submit"
class="w-full bg-blue-600 text-white py-3
rounded-lg
111         hover:bg-blue-700 transition duration-200 font-
semibold">
112         Ajouter le Livre
113     </button>
114 </div>
115 </form>
116 </div>
117
118 <!-- Liste des livres -->
119 <div class="bg-white rounded-lg shadow-md p-6">
120     <h2 class="text-2xl font-semibold mb-4 text-gray-800">
Mes Livres
121     </h2>
122     <div id="booksList" class="space-y-4">
123         <!-- Books will be displayed here -->
124     </div>
125 </div>
126 </div>
127
128 <script src="app.js"></script>
129 </body>
130 </html>

```

Listing 1 – Formulaire d'enregistrement des livres (index.html)

2.3 Capture d'écran du formulaire

The screenshot shows a web browser at localhost:3000 displaying the 'Book Reading Tracker' application. The interface is divided into three main sections:

- Statistiques Globales:** A summary section with three cards: 'Total de livres' (1), 'Livres terminés' (0), and 'Pages totales lues' (3).
- Ajouter un Livre:** A form for adding a new book. It includes input fields for 'Titre *', 'Auteur *', 'Nombre de pages *', 'Pages lues', 'Statut *' (with a dropdown menu showing 'Want to read'), 'Format *' (with a dropdown menu showing 'Print'), 'Prix (€)', and 'Suggéré par'. A blue button labeled 'Ajouter le Livre' is at the bottom.
- Mes Livres:** A section displaying a list of books. The first book shown is 'Nassima' by Nassima Rhamouch, in PDF format, priced at 0.2€, with 3/5 pages read. It has a progress bar at 60% and a 'Suivre à jour' button.

FIGURE 1 – Formulaire d'ajout de livre avec TailwindCSS

3 Question 2 : Classe Book et ses méthodes

3.1 Énoncé

Créer une classe Book avec les méthodes suivantes :

- Un constructeur
- `currentlyAt()` : Méthode pour gérer la progression de lecture
- `deleteBook()` : Méthode pour supprimer un livre

La classe Book doit être dans son propre module.

3.2 Implémentation

```
1 export enum BookStatus {
2   Read = "Read",
3   ReRead = "Re-read",
4   DNF = "DNF",
5   CurrentlyReading = "Currently reading",
6   ReturnedUnread = "Returned Unread",
7   WantToRead = "Want to read"
8 }
9
10 export enum BookFormat {
11   Print = "Print",
12   PDF = "PDF",
13   Ebook = "Ebook",
14   AudioBook = "AudioBook"
15 }
16
17 export class Book {
18   title: string;
19   author: string;
20   numberOfPages: number;
21   status: BookStatus;
22   price: number;
23   pagesRead: number;
24   format: BookFormat;
25   suggestedBy: string;
26   finished: boolean;
27
28   constructor(
29     title: string,
30     author: string,
31     numberOfPages: number,
32     status: BookStatus,
33     price: number,
34     pagesRead: number,
35     format: BookFormat,
36     suggestedBy: string,
37     finished: boolean = false
38   ) {
39     this.title = title;
40     this.author = author;
41     this.numberOfPages = numberOfPages;
42     this.status = status;
43     this.price = price;
```

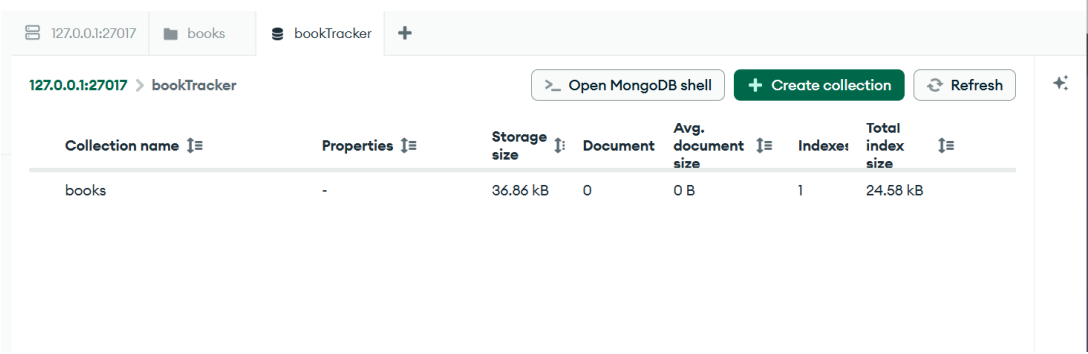
```

44     this.pagesRead = Math.min(pagesRead, numberOfPages);
45     this.format = format;
46     this.suggestedBy = suggestedBy;
47     this.finished = finished;
48
49     // Mise a jour automatique de finished quand les pages lues
50     // egales au nombre total de pages
51     if (this.pagesRead === this.numberOfPages) {
52         this.finished = true;
53     }
54 }
55
56 // Methode pour calculer le pourcentage de lecture
57 currentlyAt(): number {
58     return Math.round((this.pagesRead / this.numberOfPages) * 100);
59 }
60
61 // Methode pour supprimer un livre
62 deleteBook(): void {
63     console.log('Le livre "${this.title}" a ete supprime');
64 }
65
66 // Methode pour mettre a jour les pages lues
67 updatePagesRead(pages: number): void {
68     this.pagesRead = Math.min(pages, this.numberOfPages);
69     if (this.pagesRead === this.numberOfPages) {
70         this.finished = true;
71     }
72 }
73 }

```

Listing 2 – Classe Book (Book.ts)

3.3 Capture d'écran du code



Collection name	Properties	Storage size	Document	Avg. document size	Indexes	Total index size
books	-	36.86 kB	0	0 B	1	24.58 kB

FIGURE 2 – Code source de la classe Book

4 Question 3 : Page de suivi de lecture

4.1 Énoncé

Créer une page web permettant de suivre la lecture en listant les livres et en affichant :

- Le pourcentage de lecture pour chaque livre
- Une section globale montrant :
 - Le nombre total de livres
 - Le nombre de livres terminés
 - Le nombre total de pages lues

4.2 Description de l'implémentation

La page de suivi comprend deux sections principales :

Section des statistiques globales : Cette section affiche en temps réel les statistiques de lecture de l'utilisateur avec trois cartes colorées montrant le nombre total de livres, les livres terminés et le nombre total de pages lues.

Section d'affichage des livres : Chaque livre est affiché sous forme de carte contenant :

- Les informations du livre (titre, auteur, format, statut)
- Une barre de progression visuelle indiquant le pourcentage de lecture
- Le nombre de pages lues sur le total
- Des boutons pour mettre à jour ou supprimer le livre

4.3 Capture d'écran - Statistiques globales

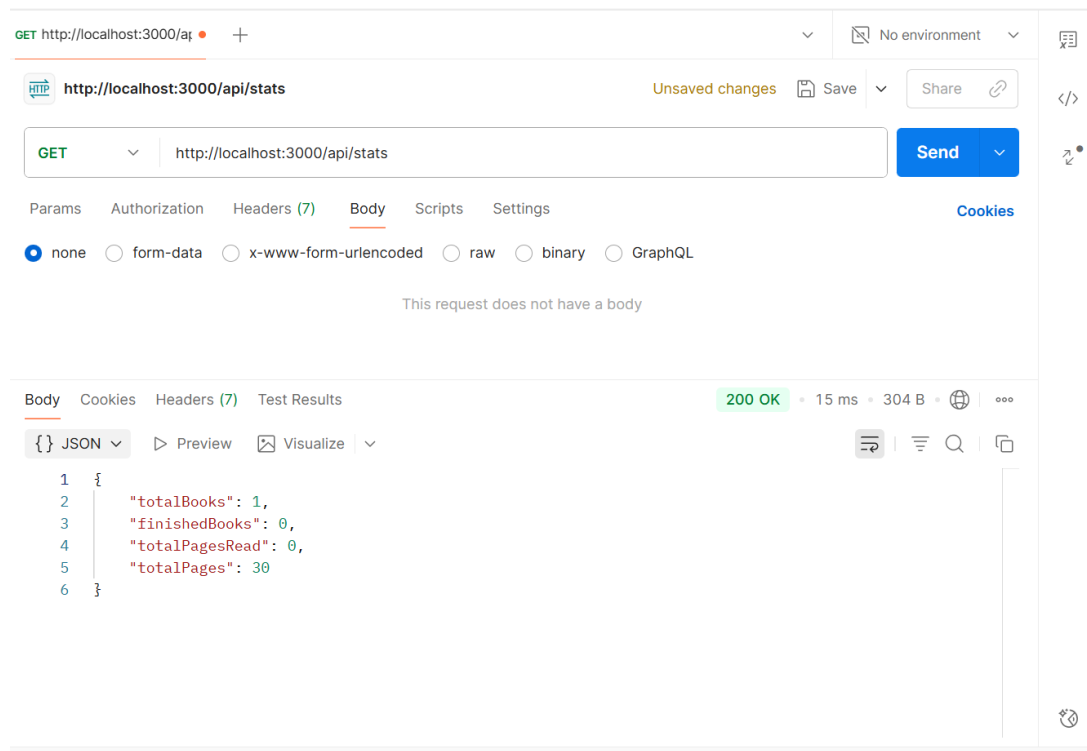


FIGURE 3 – Section des statistiques globales

4.4 Capture d'écran - Affichage des livres

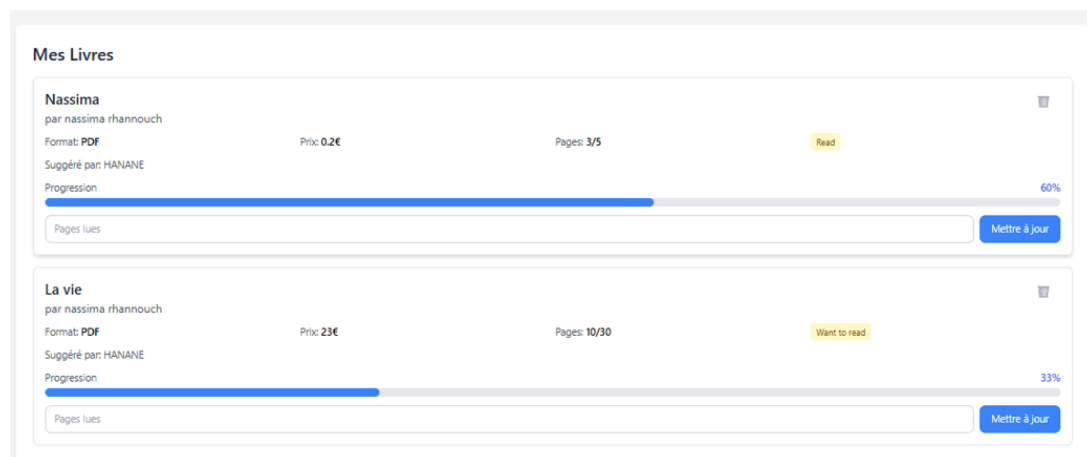


FIGURE 4 – Liste des livres avec pourcentage de lecture

5 Question 4 : Intégration avec MongoDB

5.1 Énoncé

Les livres sont stockés dans MongoDB.

5.2 Implémentation

```
1 import express, { Request, Response } from 'express';
2 import { MongoService } from './mongoService';
3 import { Book, BookStatus, BookFormat } from './Book';
4
5 const app = express();
6 const PORT = 3000;
7
8 app.use(express.json());
9 app.use(express.static('public'));
10
11 // Initialisation du service MongoDB
12 const mongoService = new MongoService(
13   'mongodb://127.0.0.1:27017',
14   'bookTracker'
15 );
16 mongoService.connect('bookTracker').catch(console.error);
17
18 // Route GET: Recuperer tous les livres
19 app.get('/api/books', async (req: Request, res: Response) => {
20   try {
21     const books = await mongoService.getAllBooks();
22     res.json(books);
23   } catch (error) {
24     res.status(500).json({ error: 'Echec de recuperation des livres' });
25   }
26 });
27
28 // Route POST: Ajouter un nouveau livre
29 app.post('/api/books', async (req: Request, res: Response) => {
30   try {
31     const {
32       title,
33       author,
34       numberOfPages,
35       status,
36       price,
37       pagesRead,
38       format,
39       suggestedBy,
40       finished
41     } = req.body;
42
43     const book = new Book(
44       title,
45       author,
46       numberOfPages,
47       status as BookStatus,
48       price,
```

```
49     pagesRead,
50     format as BookFormat,
51     suggestedBy,
52     finished
53   );
54
55   await mongoService.addBook(book);
56   res.status(201).json({ message: 'Livre ajoute avec succes', book });
57 } catch (error) {
58   res.status(500).json({ error: 'Echec d\'ajout du livre' });
59 }
60 });
61
62 // Route PUT: Mettre a jour un livre
63 app.put('/api/books/:title', async (req: Request, res: Response) => {
64   try {
65     const { title } = req.params;
66     const updatedData = req.body;
67
68     // Mise a jour automatique du statut finished
69     if (updatedData.pagesRead && updatedData.numberOfPages) {
70       if (updatedData.pagesRead >= updatedData.numberOfPages) {
71         updatedData.finished = true;
72       }
73     }
74
75     await mongoService.updateBook(title, updatedData);
76     res.json({ message: 'Livre mis a jour avec succes' });
77   } catch (error) {
78     res.status(500).json({ error: 'Echec de mise a jour du livre' });
79   }
80 });
81
82 // Route DELETE: Supprimer un livre
83 app.delete('/api/books/:title', async (req: Request, res: Response) => {
84   try {
85     const { title } = req.params;
86     await mongoService.deleteBook(title);
87     res.json({ message: 'Livre supprime avec succes' });
88   } catch (error) {
89     res.status(500).json({ error: 'Echec de suppression du livre' });
90   }
91 });
92
93 // Route GET: Obtenir les statistiques
94 app.get('/api/stats', async (req: Request, res: Response) => {
95   try {
96     const books = await mongoService.getAllBooks();
97     const stats = {
98       totalBooks: books.length,
99       finishedBooks: books.filter(b => b.finished).length,
100       totalPagesRead: books.reduce((sum, b) => sum + b.pagesRead, 0),
101       totalPages: books.reduce((sum, b) => sum + b.numberOfPages, 0)
102     };
103     res.json(stats);
104   } catch (error) {
105     res.status(500).json({ error: 'Echec de recuperation des
    statistiques' });
106   }
107 });
```

```
106   }  
107 });  
108  
109 app.listen(PORT, () => {  
110   console.log('Serveur démarre sur http://localhost:${PORT}');  
111 });  
112  
113 process.on('SIGINT', async () => {  
114   await mongoService.close();  
115   process.exit(0);  
116 });
```

Listing 3 – Serveur Express avec MongoDB (server.ts)

5.3 Captures d'écran - Connexion MongoDB

```
PS C:\Users\Lenovo\Downloads\book-reading-tracker> node dist/server.js
Server running on http://localhost:3000
Connected to MongoDB
```

FIGURE 5 – Connexion réussie à MongoDB - Terminal

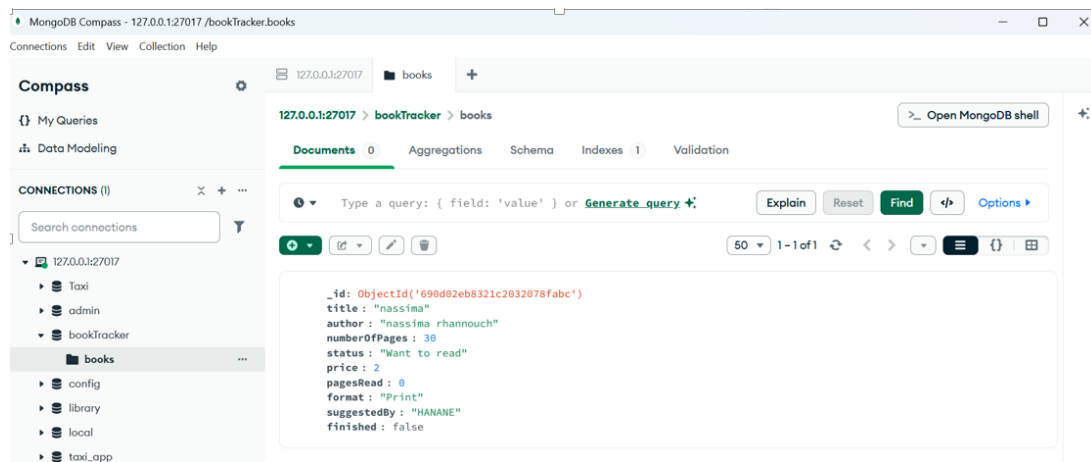


FIGURE 6 – Serveur démarré avec succès

6 Tests avec Postman

6.1 Introduction aux tests

Postman est un outil permettant de tester les API REST. Nous l'avons utilisé pour valider toutes les routes de notre application Book Reading Tracker.

6.2 Tests des différentes routes

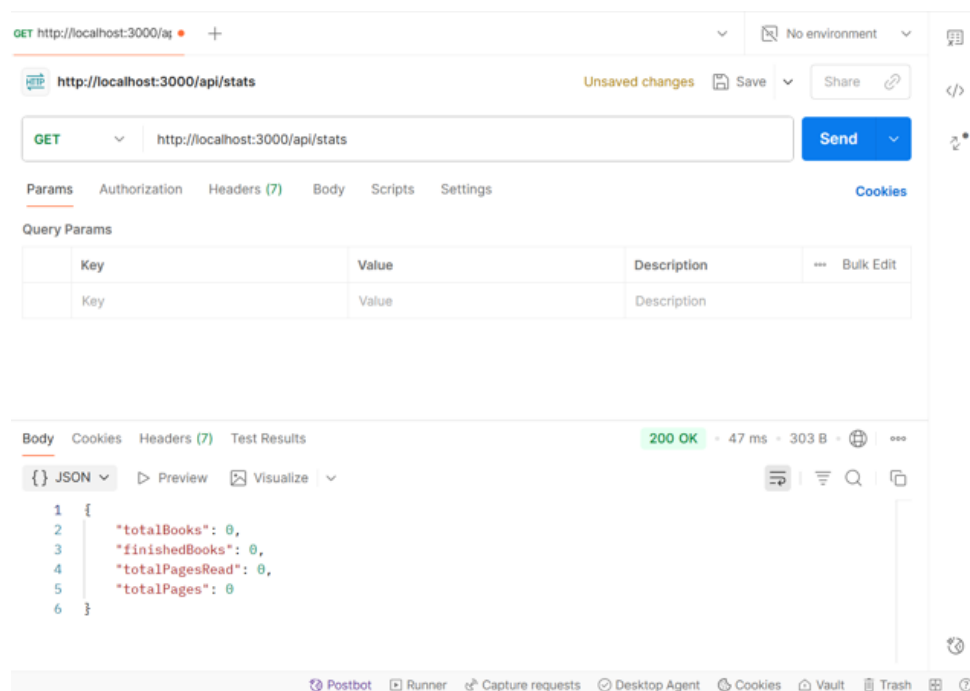
6.2.1 Test 1 : GET /api/books - Récupérer tous les livres

Objectif : Vérifier que la route retourne correctement la liste de tous les livres stockés dans MongoDB.

Requête :

- Méthode : GET
- URL : `http://localhost:3000/api/books`
- Headers : Content-Type : `application/json`

Résultat attendu : Code 200 avec un tableau JSON contenant tous les livres.



6.2.2 Test 2 : POST /api/books - Ajouter un nouveau livre

Objectif : Vérifier que l'ajout d'un nouveau livre fonctionne correctement.

Requête :

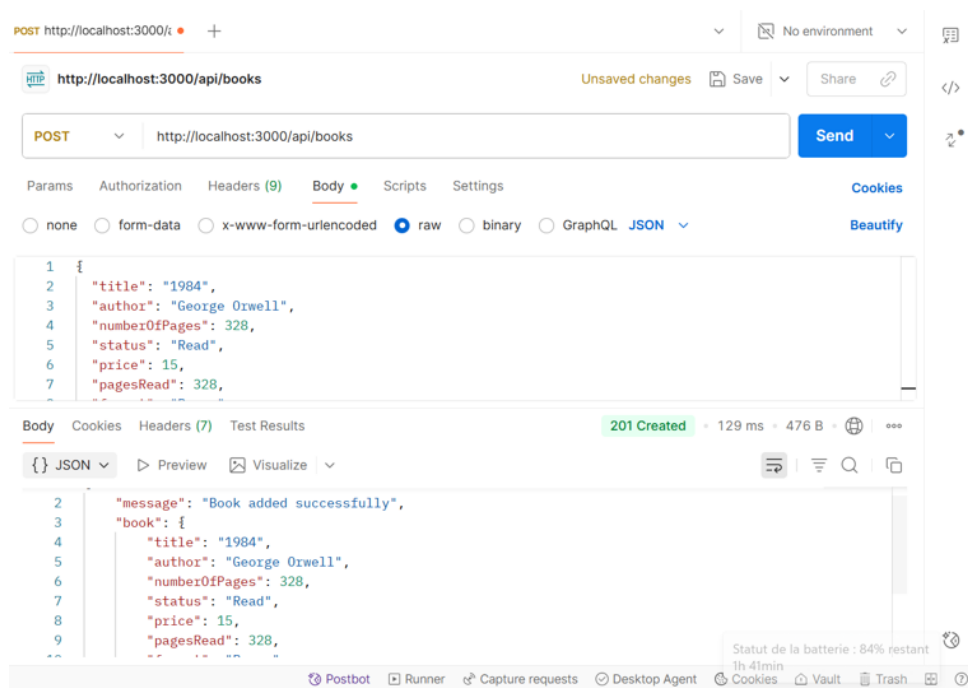
- Méthode : POST
- URL : `http://localhost:3000/api/books`
- Headers : Content-Type : `application/json`
- Body (raw JSON) :

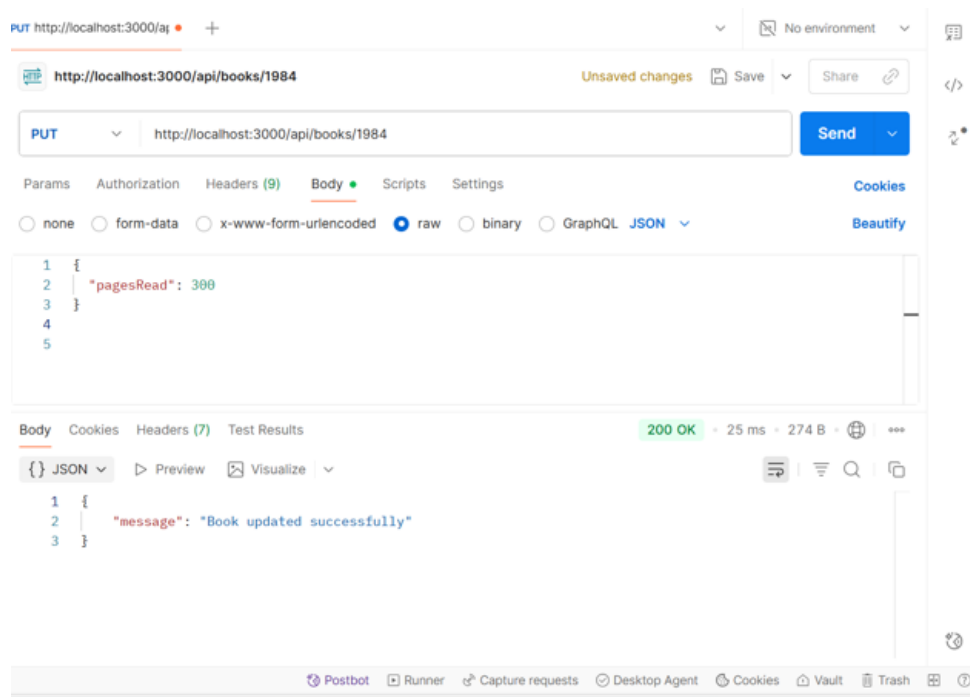
```
1 {
2   "title": "1984",
```

```
3  "author": "George Orwell",
4  "numberOfPages": 328,
5  "status": "Read",
6  "price": 15,
7  "pagesRead": 328,
8  "format": "Print",
9  "suggestedBy": "Marie",
10 "finished": false
11 }
```

Listing 4 – Corps de la requête POST

Résultat attendu : Code 201 avec le message de confirmation et l'objet livre créé.





6.2.3 Test 3 : PUT /api/books/ :title - Mettre à jour un livre

Objectif : Vérifier que la mise à jour d'un livre existant fonctionne, notamment la mise à jour automatique du statut `finished`.

Requête :

- Méthode : PUT
- URL : `http://localhost:3000/api/books/Le%20Petit%20Prince`
- Headers : Content-Type : `application/json`
- Body (raw JSON) :

```
1 {
2   "pagesRead": 300,
3 }
4 }
```

Listing 5 – Corps de la requête PUT

Résultat attendu : Code 200 avec message de confirmation. Le champ `finished` doit automatiquement passer à `true` car `pagesRead = numberOfPages`.

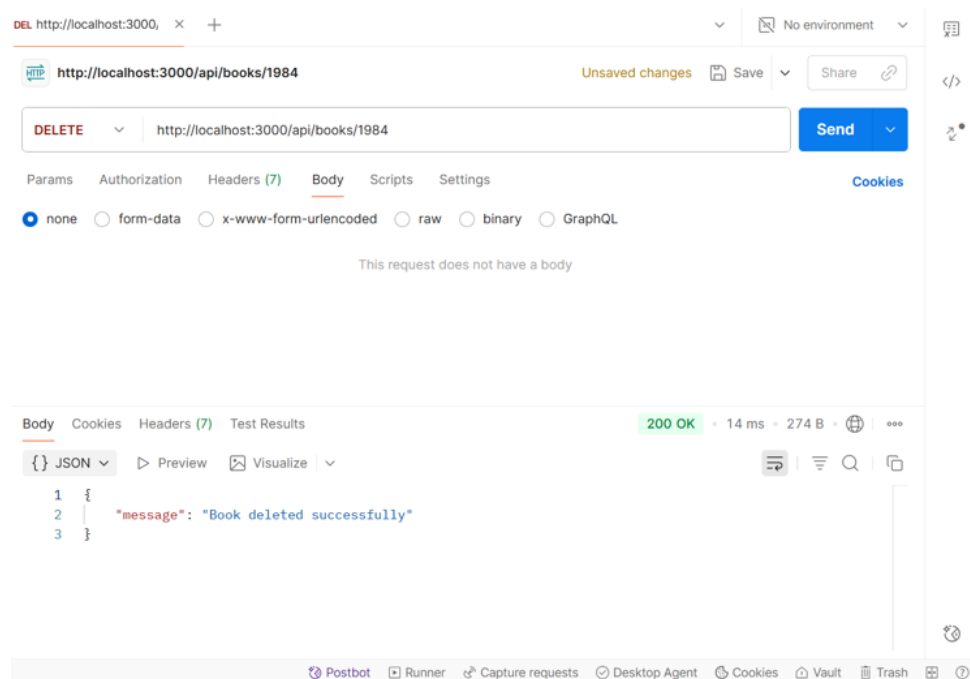
6.2.4 Test 4 : DELETE /api/books/ :title - Supprimer un livre

Objectif : Vérifier que la suppression d'un livre fonctionne correctement.

Requête :

- Méthode : DELETE
- URL : `http://localhost:3000/api/books/Le%20Petit%20Prince`
- Headers : Content-Type : `application/json`

Résultat attendu : Code 200 avec message de confirmation de suppression.



6.2.5 Test 5 : GET /api/stats - Récupérer les statistiques

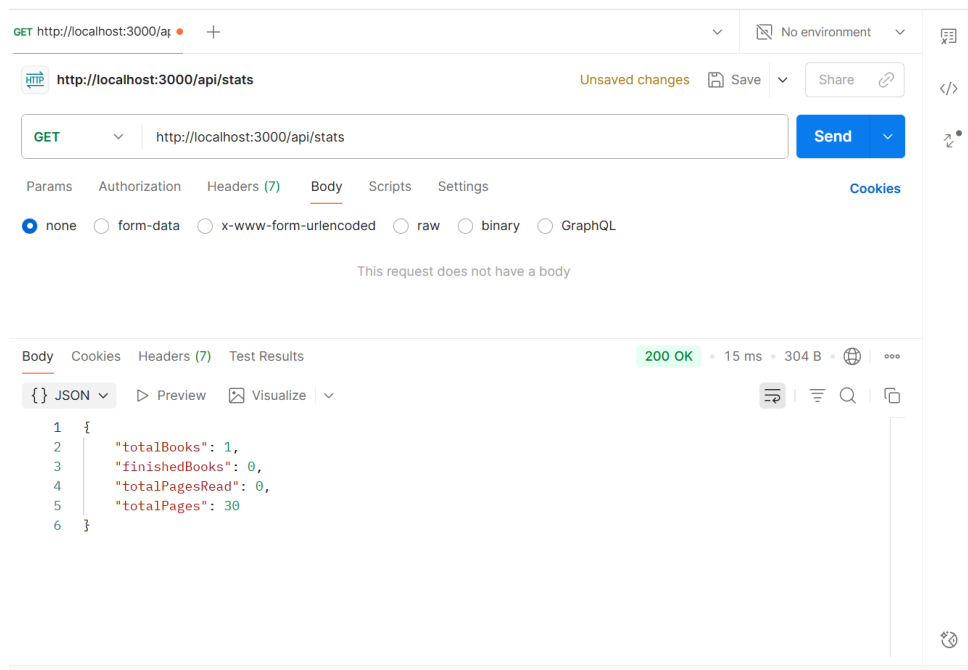
Objectif : Vérifier que les statistiques globales sont calculées correctement.

Requête :

- Méthode : GET
- URL : http://localhost:3000/api/stats
- Headers : Content-Type : application/json

Résultat attendu : Code 200 avec un objet JSON contenant :

- totalBooks : nombre total de livres
- finishedBooks : nombre de livres terminés
- totalPagesRead : nombre total de pages lues
- totalPages : nombre total de pages de tous les livres



7 Conclusion

Ce travail pratique a permis de développer une application web complète et fonctionnelle de suivi de lecture de livres. Le projet a été réalisé en utilisant des technologies modernes et des bonnes pratiques de développement.